

1. INVERTED INDEX & TF-IDF

WHAT: Term → [(docID, tf), ...] + df per term

WHY: Fast lookup — find docs containing query terms

WHEN: Always — foundation of all retrieval

TF-IDF weight:
 $w = tf \times \log_2(N/df)$ [IDF=0 if df=N, term in ALL docs!]

EXAM TRAP:

If a term appears in ALL docs → IDF=log(N/N)=0 → contributes NOTHING
Check df=N before computing scores!

2. VECTOR SPACE MODEL (VSM)

WHY: Rank by angle between doc & query vectors

WHEN: Similarity ranking with TF-IDF weights

Cosine similarity:
 $\cos(D, Q) = (D \cdot Q) / (|D| \times |Q|)$ where $|v| = \sqrt{\sum v_i^2}$

Step-by-step:

- 1. Compute dot product: $\sum (d_i \times q_i)$ for matching terms only
- 2. Compute |D| and |Q| (sqrt of sum of squares of ALL term weights)

3. EVALUATION METRICS

Precision / Recall / F1:
 $P = |Rel \cap Ret| / |Ret|$ $R = |Rel \cap Ret| / |Rel|$ $F1 = 2PR / (P + R)$

Average Precision:
 $AP = (1/|R|) \times \sum P(k)$ at relevant ranks ← DIVIDE BY TOTAL RELEVANT

DCG / nDCG (log2 values given in Q):
 $DCG@k = \sum rel_i / \log_2(i+1)$ $nDCG = DCG/IDCG$

- AP: divide by TOTAL relevant (not retrieved relevant)
- R-P graph: rank $k = k / P(\text{recall} = k/N, \text{rel})$ — derive ranks from graph
- Interpolated $P(r) = \text{MAX precision at ANY recall} \geq r$
- Non-interp can go UP (relevant found after non-relevant)
- nDCG: IDCG = ideal ranking (highest grades first)
- $P@k$: count relevant in top-k, divide by k

4. HEAP'S LAW (VOCABULARY GROWTH)

Formula:
 $V_n = K \times n^b$ (b=0.5 typical) Find K & b via $\log(n)$
Sublinear: larger collections → fewer NEW terms per doc added
EXAM: verify K, then $V_{\text{new}} = K \times \sqrt{n_{\text{new}}}$, $\text{new_terms} = V_{\text{new}} - V_{\text{old}}$

5. ROCCHIO RELEVANCE FEEDBACK

WHY: Improve query using user feedback on results

WHEN: User marks docs relevant/non-relevant

Rocchio formula (on cribsheet):
 $q = \frac{1}{|D_r|} \sum_{d \in D_r} d + \lambda \frac{1}{|D_{nr}|} \sum_{d \in D_{nr}} d$ (normalized)

EXAM STEPS:

- 1. q_{orig} : raw TF of query terms
- 2. Relevant centroid = avg of relevant doc vectors
- 3. Non-rel centroid = avg of non-relevant doc vectors
- 4. Apply formula. Set NEGATIVE weights to 0

6. LANGUAGE MODELS + JM SMOOTHING

WHY: Zero-probability fix — every term gets non-zero prob

WHEN: Rank docs by $P(\text{query}|\text{doc})$ — missing terms scored 0 without smoothing

JM Smoothing (on cribsheet), $L = \text{lambd}$:
 $P(w|d) = \frac{(1-L) \cdot \text{tf}(w, d) / |d| + L \cdot P(w|C)}$

Score a document:

- 1. $P(w|d)$ PRODUCE over all query terms of $P(w|d)$
- Rare terms (low $P(w|C)$) discriminate more — doc with high tf of rare term wins
- $\text{Lambda} = 0.5$: equal weight to doc-specific + collection probability

7. BINARY INDEPENDENCE MODEL (BIM)

WHY: Probabilistic ranking — $P(\text{relevant}|\text{doc}, \text{query})$

WHEN: Probabilistic IR; term presence/absence only (binary)

RSV formula (only sum where $d_i = 1$):
 $RSV = \sum_{d_i=1} \log_2 \left(\frac{1 + p_i}{1 - p_i} \right) / \sum_{d_i=1} p_i$ for terms where $d_i = 1$

With relevance (R known):

$p_i = (r_i + 0.5) / (R + 1)$ $s_i = (df - r_i + 0.5) / (N - R + 1)$

! LIMITATIONS: binary only, ignores tf, relevant doc missing term=RSV 0

8. ZIPF'S LAW

Formula:
Zipf's Law: $P(w) = A / w^B$ (B=2 typical) Then $P(w_1) = A/1$, $P(w_2) = A/2$
Top 4 words cover ~20% of all tokens (Zipf skew)

9. HITS ALGORITHM

WHY: Topic-specific authority — 2 scores per page
WHEN: Web IR, link analysis for a query sub-graph
Hub = links to many good authorities
Authority = linked by many good hubs

Update rules (iterate until convergence):
$$a(p) = \sum_q b(q) \text{ for all } q \rightarrow p, \text{ and } b(q) = \sum_p a(p) \text{ for all } p \rightarrow q$$

- 1. Build adjacency matrix A ($A[i][j]=1$ if $i \rightarrow j$)
- 2. Authority = principal eigenvector of M , $a = A^T * A$
- 3. Hub = principal eigenvector of M , $h = A * A^T$
- 4. Diagonal of M , $a[i][i]$ = in-degree of page i (verify!)

! EXAM STEPS: Build $A \rightarrow$ transpose $\rightarrow$ multiply $\rightarrow$ write eigenvalue eqns
• Normalise each iteration by max value (or L2 norm)

10. TAAT VS DAAT TRAVERSAL

TAAT (Term-At-A-Time):
• Process one FULL posting list before next term
• Memory: $O(N)$ — accumulator for every doc in corpus

DAAT (Document-At-A-Time):
• Process all terms for ONE doc, then move to next doc
• Memory: $O(K)$ — only top-K heap + pointers. BETTER!
• MaxScore: skip docs if max_possible_score < current threshold

11. IR MODEL ASSUMPTIONS (ALWAYS 4 MARKS)

- 1. Term Independence (BoW): "New York" != "New" + "York"
 - 2. Document Independence: ignores diversity/novelty in results
 - 3. Static relevance: ignores time (2019 COVID doc misleading in 2026)
 - 4. Topical = User relevance: ignores readability/credibility
- For each assumption — always give a CONCRETE EXAMPLE!**
e.g. Term independence: "Apple" query returns recipe docs due to shared word with Apple Inc. — bag of words cannot distinguish context

12. INDEX COMPRESSION

Lossless:
d-gaps, VByte, Gamma codes, PForDelta
Measure: compression ratio + decompression speed. MAP unchanged.
Lossy:
Static pruning, stopword removal, quantisation
Measure: compression ratio AND MAP/nDCG (effectiveness may drop)

13. NEURAL IR — ARCHITECTURES

Bi-Encoder (Dense Retrieval):
• Encodes q and d SEPARATELY into dense vectors
• Docs pre-encoded offline $\rightarrow$ fast ANN search at query time
• Use as FIRST STAGE — finds semantic matches (no lexical overlap)
Cross-Encoder:
• Encodes (q,d) JOINTLY — full token interaction
• Better quality but cannot pre-encode $\rightarrow$ slow at query time
• Use as RE-RANKER only (top 100 from first stage)

ColBERT / MaxSim (Late Interaction):

MaxSim (on cribsheet):
• Token-level matching, doc tokens pre-computed offline — best of both
$$\text{MaxSim}(Q,D) = \sum \text{over } q \text{ of: } \max \text{ over } d \text{ of } \text{sim}(q,d)$$

14. PYTERRIER PIPELINES (EXAM: 7-16 MARKS!)

Operators (on cribsheet):
PRF Pipeline (baseline): $\text{base} \rightarrow \text{core} \rightarrow \text{rewriter} \rightarrow \text{index}$
• rewriter1 sees top-3 docs + index. Uses Bo1/RM3 (PRF)
• rewriter2 sees ONLY query. Uses thesaurus/word2vec

Experiment design — always justify:
`pt.Experiment([baseline, system], topics, qrels, eval_metrics=["ndcg_cut_10", "map", "recall_1000"], baseline=0)` ← enables paired t-test auto
! NaN p-value: re-ranker does NOT change candidate set $\rightarrow$ Recall@1000 identical on ALL queries $\rightarrow$ variance=0 $\rightarrow$ t-test = 0/0 = NaN (correct behaviour!)

15. DENSE RETRIEVAL + PASSAGE AGGREGATION

PROBLEM: BERT 512-token limit; single vector loses long-doc content
FIX: Split into overlapping passages, encode each separately
MaxPassage: score = BEST passage score
Good when ONE strong section proves relevance
SumPassage: score = SUM of all passage scores
Better when content SPREAD throughout (e.g. review articles)

16. EXAM STRATEGY (1H 30MIN)

- Q1: ~20 min** Inverted index + TF-IDF + Cosine + P/R/AP/DCG
 - Q2: ~20 min** [Blank]
 - Q3: ~20 min** [Blank]
- QUICK WINS: show all working! partial credit given for each step**
• Write formula $\rightarrow$ substitute values $\rightarrow$ compute $\rightarrow$ state result
• Verify: IDF=0 check, divide AP by TOTAL relevant, normalise HITS